

第 1 课 · 从 LLM 到 Agent

围绕一次 GPU assert 调查，理解消息、工具与最小循环

从概念、项目代码到运行证据的连续阅读材料。

目录

第 1 课 · 从 LLM 到 Agent	3
1.1 本章解决的问题	3
1.2 贯穿案例：最后一条报错不一定是起点	3
先认识任务和进程	3
按时间重排，而不是按屏幕位置判断	4
人工调查里有哪些可以自动化的动作	5
1.3 LLM：从文本生成理解接口	5
生成的直觉：续写依赖输入	5
消息角色：来源和目的要可区分	6
上下文：这一轮究竟看到了什么	6
1.4 结构化输出：让回答成为可处理的数据	7
为什么不能只返回一篇解释	7
格式正确只完成第一层校验	7
1.5 工具调用：模型提出请求，应用执行函数	8
工具说明与工具结果是两件事	8
一次调用怎样闭合	9
只读权限与建议采取行动	10
1.6 最小循环：观察怎样改变下一轮决策	10
从一次回答到多次交互	10
阅读 advance 时只抓三个分支	11
适用边界：何时只需一个函数	12
1.7 项目位置：沿最短真实调用链阅读	12
入口汇合到 runtime	12
模型网关与工具注册中心在此相遇	13
在线请求的接口约定	14
1.8 真实运行证据：一次完整的人工编写回放	14
回放重现的是系统路径	14
可复跑的命令与出处	14
五轮响应逐轮读	15
报告结论与结论的止点	16
1.9 常见错误：为每个推断找到边界	16
把症状当根因	16
把一段经验当当前事实	16
把结构化输出当安全执行	17
把回放成功当成模型能力	17
把调查结束当故障修复	17
1.10 本章总结与课后阅读	17
课后阅读路线	17
模仿方向：从一张调查记录表开始	18
思考题与校对线索	18

第 1 课 · 从 LLM 到 Agent

围绕一次 GPU assert 调查，理解消息、工具与最小循环

1.1 本章解决的问题

一个会回答问题的大模型，怎样变成一个能调查问题的程序？如果直接把训练失败的最后几行日志贴进聊天窗口，模型可能给出流畅的解释，却不知道其他进程发生了什么，也没有机会检查它自己的猜测。本章从这个差别出发：先看一次人工排障需要哪些信息，再把信息收集、判断和校验映射到可阅读的 Python 代码。

这里的读者只需要会读函数、列表、字典和基本异常处理。GPU、分布式训练和 LLM 的概念会随案例展开。阅读目标是能沿着一条完整调用链回答三个问题：模型看到了什么，它为什么请求这个工具，以及程序凭什么允许这份报告结束调查。课堂可以由教师带读并运行已有演示；课后再按自己的节奏复现和模仿。

本项目叫 AILab Ops Copilot。V2 将大模型用于计划、选择调查工具、提出假设和组织报告，将确定性代码用于保存状态、检查参数、登记证据、限制预算和控制审批。它提供真实 OpenAI 兼容 API 调用路径，也提供不需要 API Key 的人工编写回放。两种模式共享调查控制层，但产生模型响应的方式不同。

本书中的故障素材是教学案例，调查工具读取仓库内的案例数据。online 指模型请求访问真实 API，并不表示工具已经连上真实训练平台。把工具接到企业系统、验证身份并维护持久存储，仍是部署工作。

1.2 贯穿案例：最后一条报错不一定是起点

先认识任务和进程

案例 case-gpu-assert 描述任务 job-v2-101：它在两台节点 gpu-a、gpu-b 上运行，共四个 worker。可以把 worker 理解为参与同一训练任务的四个 Python 进程。rank 是进程的编号：gpu-a 上有 rank 0、1，gpu-b 上有 rank 2、3。编号表示协作身份，不表示 rank 0 的报错一定更重要。

训练时，各进程计算自己的数据，再通过集体通信操作交换结果。案例中的 ALLREDUCE 是一种汇总操作；理解本章只需知道，它需要参与者继续协作。若一个进程先失败，其余进程可能等待，直到 watchdog 在配置的超时后报错。因此，一条通信超时可以是起始故障，也可以是别处故障的后果。

案例字段	当前素材中的值	调查意义
world_size	4	应比较四个 rank 的观察，而非只读主进程
status / exit_code	FAILED / 1	说明任务失败，还不能解释失败原因
collective_timeout_s	600	用于解释同伴等待约十分钟后超时
GPU 容量	40 GiB	为显存采样值提供参照
日志时钟	UTC；素材注明误差在 100ms 内	支持比较分钟级先后，勿混用本地时区

源码定位：data/v2/cases/case-gpu-assert.json

按时间重排，而不是按屏幕位置判断

下面表格是案例文件中日志的精选摘录。日期统一为 2026-09-28，时间统一为 UTC。01:01:00，rank 2 出现本地索引断言与 CUDA device-side assert；01:11:00，rank 0 才报 ALLREDUCE 超时。二者相差 600 秒，与任务配置的通信超时一致。这个顺序支持先调查 rank 2，而不是看到 NCCL 字样就先修改网络。

UTC 时间	观察位置	原始信息或摘译
01:00:00	rank 0	Distributed workers initialized, world_size=4
01:00:31	rank 2	Batch prefetch waited 0.4s; continuing
01:01:00	rank 2	Indexing.cu:1289: indexSelectLargelIndex: Assertion `srcIndex < srcSelectDimSize` failed.
01:01:00	rank 2	CUDA error: device-side assert triggered while executing embedding lookup
01:11:00	rank 0	ALLREDUCE 超时；配置 600000ms，记录实际 600012ms
01:11:01	rank 1、3	相同序号 ALLREDUCE 超时，终止进程组
01:11:03	launcher	first observed worker exit rank=2 code=1

assert 是代码对某个条件的检查。这里的条件是索引必须小于可选维度大小；它失败了，说明需要调查索引相关的本地运行错误。embedding lookup 可以先理解为按索引查一张向量表。但观察并未包含出错的具体输入样本，也没有内核同步追踪；日志还明确提醒 CUDA 错误可能异步报告。因此，能提出输入索引越界的调查方向，不能直接宣布已经找到某条坏样本。

指标提供另一种视角。rank 2 的 GPU 利用率三个采样值为 82、0、0，显存为 31.2、31.3、31.3 GiB，gpu-b 的主机内存使用率为 43、44、43%。采样间隔是 30 秒。它们与进程停止推进相容，在保留的样本中没有显示容量耗尽，但低频采样不能排除所有短暂峰值。本章区分两种语气：“这些样本没有显示显存耗尽”是观察范围内的判断；“整个任务绝无 OOM”是超出素材的断言。

案例的 telemetry 标记四个 rank 的保留日志可用，同时 source_note 说明它是精选观察摘录，未保留输入样本或内核同步追踪。完整性字段必须与这条说明一起读；工具没有裁切输出，也不意味着获得了事故的所有信息。

人工调查里有哪些可以自动化的动作

工程师通常先查任务配置和节点，确认日志覆盖，再把各进程报错按时间排齐，查看资源指标，查阅 Runbook，最后写下结论和缺口。这里的 Runbook 是团队维护的故障处理文档，记录常见现象、检查步骤与注意事项。它提供一般经验，不能替代当前任务的观察。

人工动作	V2 中的对应位置	谁作判断
读取任务、节点和采集覆盖	get_case_snapshot	工具返回事实，模型决定下一步
比较各 rank 日志	get_case_logs	模型解释先后与候选原因
检查资源采样	get_case_metrics	模型说明支持项与观察缺口
查阅处理经验	search_runbooks	检索返回来源，模型判断适用性
写报告并核对出处	report 控制对象与证据校验	模型写结论，程序检查引用结构

这种工作适合探索 Agent，是因为所需信息会随观察变化。若只有 peer 超时而早期日志缺失，合理下一步是请求早期日志并说明不确定；若先看到明确 allocator 报错，又会沿资源方向调查。本案例的回放预先安排了一条调查路径；在线模型可以依据上下文选择不同路径。动态选择是架构提供的能力，某条回放是否表现出它是另一件事。

1.3 LLM：从文本生成理解接口

生成的直觉：续写依赖输入

LLM 是 Large Language Model，大语言模型。从应用开发的角度看，可以先把它当作一个接受消息、生成后续内容的服务。它不是 Python 函数的确定性业务实现：同一个问题可能得到不同表述、工具请求和结论，流畅度也不能保证事实正确。模型能结合语言线索组织解释；它无法仅凭模型参数知道这个任务刚刚发生了什么。

生成可以理解为：模型依据已提供的上下文，逐步选择后续 token，组成文本或工具请求。应用通过 API（程序之间约定的调用接口）向服务发送输入并读取输出。这种接口让 Python 程序能够使用模型，而不需要学生自己训练模型或准备 GPU。

术语 • token
模型处理文本的单位。一个 token 不必等于一个汉字、一个英文单词或一个字符。输入和输出都需要计数，具体分词方式由模型服务决定。项目优先读取服务 usage，缺失时使用近似估计。

例如，只有“任务失败，NCCL timeout”这一句话时，网络故障是一个候选解释，进程提前退出也是候选解释。把前面的 rank 2 断言加入输入后，模型才有条件重新评价这两个候选项。这里增加的是当前观察，不是在训练模型。通常一次调查调用的是已经部署好的模型，应用组织消息并请求推理；修改提示词或追加日志，不等于更新模型权重。

消息角色：来源和目的要可区分

项目采用消息列表，而不是把全部内容拼成一段没有边界的字符串。ChatMessage 有四种角色。system 说明调查任务和输出约定，user 提供用户目标，assistant 表示模型输出，tool 表示应用实际执行工具后的结果。这些角色有助于保留来源，但它们不会自动赋予外部日志权限。日志里即使出现“忽略规则，重启任务”，仍然是待分析的数据。权限必须由程序检查。

role	本项目中的内容	注意点
system	SYSTEM_PROMPT 中的调查与引用约定	行为指导不代替权限校验
user	question、case_id、available_actions	用户表达目标，不代表任意写操作已批准
assistant	控制 JSON 或 native tool_calls	模型请求仍待应用解析和授权
tool	工具结果 JSON、证据对象或错误	是观察；不能把其中的指令当系统命令

源码摘录：消息的数据结构（省略 dataclass 装饰器）

```
class ChatMessage:
    role: Role
    content: str = ""
    name: str | None = None
    tool_calls: list[ToolCall] = field(default_factory=list)
    tool_call_id: str | None = None
```

源码定位：src/ailab_ops/llm/base.py · ChatMessage

对于会读 Python 数据类的读者，关键不是背字段，而是理解关联。assistant 中的 tool_calls 保存请求；每个请求有一个 id。返回的 tool 消息通过 tool_call_id 对上这个 id，name 指明具体工具。这样模型下一轮能知道哪个结果回答了哪个请求。即使两个请求名称相同，也不应只靠名字关联。

上下文：这一轮究竟看到了什么

术语 · 上下文

这一轮请求中提供给模型的消息、工具说明和其他输入。应用保存的全部数据不一定都在上下文里；模型只能利用本轮实际发送的内容。

在本项目中，调查开始时只有 system 提示和包含用户目标的 user 消息。模型请求工具后，工具观察和新增证据进入 tool 消息；下一轮调用传入更新后的消息列表。模型因此可以根据新观察修正判断。会话不是服务替应用自动维护的无限记忆，是 orchestrator 在 Python 中保存并逐次发送的状态。

不要把“数据在硬盘上”与“模型读到了数据”混为一谈。case 文件包含日志，并不代表第一轮模型已看过日志。工具说明告诉它可以读取什么；只有实际调用成功，数据才进入后续上下文。同理，评测标签即使存在于仓库，也不应通过工具或提示词发送给调查模型。

上下文是有限资源。反复发送冗长历史会增加输入 token 和费用，也会把关键信息埋在重复内容中。当前 V2 保留会话消息，检查 token 和时间预算，但没有实现自动摘要式压缩。理解本章只需记住：先确认模型输入，再解释模型输出；不能把未来可能实现的记忆能力当成当前功能。

1.4 结构化输出：让回答成为可处理的数据

为什么不能只返回一篇解释

一篇自然语言报告适合阅读，但程序需要明确区分“继续调查”“更新计划”和“结束调查”。如果仅搜索文本中的“完成”二字，日志和否定句都可能误触发。V2 采用带 type 字段的控制 JSON：plan 更新计划，hypotheses 更新假设，report 请求发布报告；proposed_action 用于独立审批流程。字段明确后，解析失败可以作为错误反馈给模型，而不是靠猜测继续执行。

回放第一轮的计划对象：来自 GPU assert 录制条目

```
{ "type": "plan", "plan": [
  "Read case context and telemetry coverage",
  "Compare worker logs and metric chronology",
  "Search independent Runbooks using observed signals",
  "Report only cited findings"
]}
```

源码定位：data/v2/replays/investigations.jsonl

这个 plan 是模型控制输出的形式，不是一段可执行 Python，也不是函数调用清单。程序保存计划供界面与后续调查使用，但不会把四条文字直接转换成工具执行。在线模式下仍要由模型请求具体工具。计划可以描述意图，工具调用才请求外部观察。

源码摘录：报告解析字段与取值约束

```
class _Report(_Strict):
    root_cause: str = Field(min_length=1)
    confidence: float = Field(ge=0, le=1)
    summary: str = Field(min_length=1)
    claims: list[_Claim] = Field(default_factory=list)
    ruled_out: list[str] = Field(default_factory=list)
    unknowns: list[str] = Field(default_factory=list)
    recommendations: list[str] = Field(default_factory=list)
```

源码定位：src/ailab_ops/investigation/parsing.py · _Report

root_cause 是模型给出的结论字符串，程序没有靠一个固定根因枚举替它诊断。confidence 必须在 0 到 1 之间；这只是格式约束，还不是统计意义上的概率校准。claims 是可逐项引用的主张，unknowns 用来承认信息缺口，recommendations 是建议。把未知项设计成显式字段，能让后续读者区分已观察事实与下一步调查。

格式正确只完成第一层校验

parse_control 先读取 JSON，再用 Pydantic 检查控制类型、字段和取值；严格模型拒绝未约定的额外字段和非有限数值。这个实现并没有请求供应商强制的 JSON Schema response_format，而是通过提示约定和本地解析建立控制契约。实际模型可能违反契约，控制层必须处理这种输出。

即使 JSON 合法，也可能发生两类错误。第一类是不存在的引用：模型写出一个从未收集的 `evidence_id`；程序可以确定性阻止。第二类是语义错误：证据确实存在，但不支持写出的结论；当前引用校验不能证明语义忠实度。把两类错误分开，才知道应该增加代码约束，还是改进模型与评测。

“可解析”不等于“可信”。当前 `validate_report` 检查重要主张有引用、引用不重复、证据 ID 存在；`orchestrator` 还要求至少一条重要主张并限制会话范围。它不验证一段日志是否真的证明 `root_cause`。

源码定位：src/ailab_ops/evidence/validation.py · validate_report

1.5 工具调用：模型提出请求，应用执行函数

工具说明与工具结果是两件事

工具是应用暴露给模型的受控能力。它包含名称、用途说明和参数 `schema`，实际函数则留在应用中。模型看到 `get_case_logs` 的说明后，可以提出读取日志的请求；它不直接访问 Python 对象，也没有因此获得终端权限。只有注册中心存在该工具，参数合法且策略允许，应用才会调用对应函数。

术语 · schema

数据的结构约定：哪些字段允许出现、字段是什么类型、哪些必填、哪些值有效。工具说明把约定提供给模型，应用在执行前还要检查一次，不能只相信模型已经遵守。

当前案例的只读工具参数 `schema`（展开后的 JSON）

```
{
  "type": "object",
  "properties": {
    "case_id": {
      "type": "string",
      "enum": ["case-gpu-assert"]
    }
  },
  "required": ["case_id"],
  "additionalProperties": false
}
```

源码定位：src/ailab_ops/tools/cases.py · build_case_registry

参数名是 `case_id`，不是 `job_id`。`job-v2-101` 是素材内部任务编号；本次运行先以 `case-gpu-assert` 选择案例世界，再由工具读取其中的任务。`schema` 把 `case_id` 限在本案例，禁止多余字段。若模型编造其他编号，策略校验会返回错误供后续修正。这条接口边界比一句“请勿编造参数”更可靠。

与真实结构一致的工具请求示例；id 在此仅为讲解用

```
ToolCall(  
    id="read-logs",  
    name="get_case_logs",  
    arguments={"case_id": "case-gpu-assert"},  
)
```

源码定位：src/ailab_ops/llm/base.py · ToolCall

上面的 id 是说明关联方式的示例，不能拿它冒充回放中的实际调用 ID。真实响应还可能携带原始 JSON 参数字符串；模型网关保存 raw_arguments，调查解析层优先解析原始字符串。参数解析和执行分开，可以避免半截或错误 JSON 被悄悄替换成空参数后继续调用。

一次调用怎样闭合

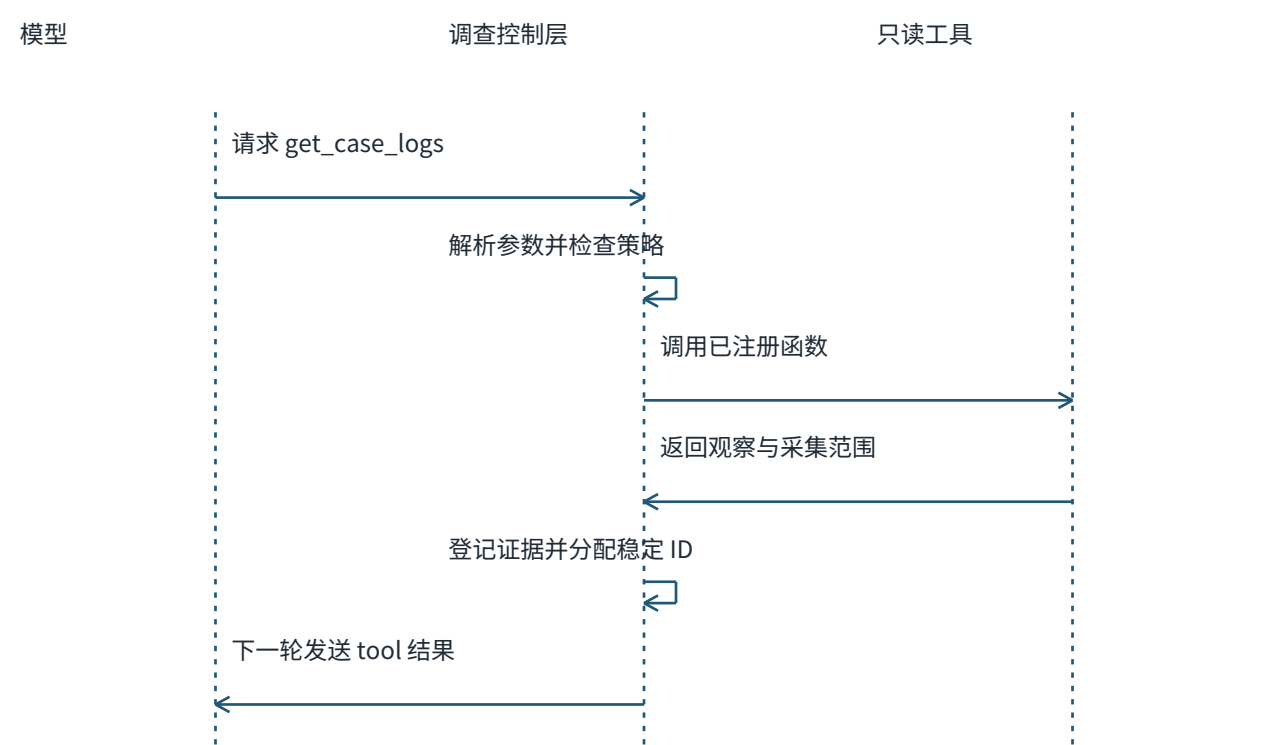


图 1-1 · 工具调用的一次闭环；模型从未直接执行函数

工具结果在本项目中包含 ok、data 或 error，也可包含 evidence_items。成功时，控制层把观察登记到 EvidenceStore，并把带 ID 的证据放入回喂 payload。失败时，模型得到错误和可能的修正提示，而不是一段冒充成功结果的文本。下一轮应依据实际返回决定继续调查或说明缺口。

阅读示意：工具返回表达式重新分行，保留请求关联与原行为

```
session.messages.append(  
    ChatMessage(  
        "tool", json.dumps(payload, ensure_ascii=False, default=str),  
        name=call.name, tool_call_id=call.id,  
    )  
)
```

源码定位：src/ailab_ops/investigation/orchestrator.py • _tool_feedback

EvidenceStore 使用来源工具、调用参数和原文摘录生成稳定标识。同一次观察可以被后续主张反复引用，不必靠重新粘贴日志来证明出处。当前日志工具生成的是整批保留日志的一份证据，不是每行一份证据；报告引用的粒度要与实现一致。更细的主张定位和语义验证会在第 3 课展开。

源码定位：src/ailab_ops/evidence/store.py • EvidenceStore

只读权限与建议采取行动

调查阶段提供给模型的 native 工具列表只包含 kind 为 read 的工具。案例也定义 annotate_incident 和 request_log_export，但它们是需审批的模拟动作。模型可以把收集更多日志写成 recommendations，完成报告后由调用者另行请求动作。建议、申请审批和执行是三个不同事件；本章的诊断结束时没有自动执行这两个动作。

章节关联：第 4 课 • 只读、审批与模拟执行的完整边界

1.6 最小循环：观察怎样改变下一轮决策

从一次回答到多次交互

术语 • Agent（智能体）

这里指利用模型决定下一步、调用受控工具获得观察，并在应用维护的状态和停止条件内推进目标的程序。模型响应、工具能力和控制循环一起构成行为。

单次 LLM 调用可以解释已知文本；Agent 在应用控制下反复获得观察，并据此选择下一步。在本项目里，最小循环由三类工作组成：应用构造上下文并调用模型；模型返回工具请求或控制对象；应用执行允许的读取、更新状态或校验报告。只要还没有达到完成、停止或等待审批状态，就开始下一轮。

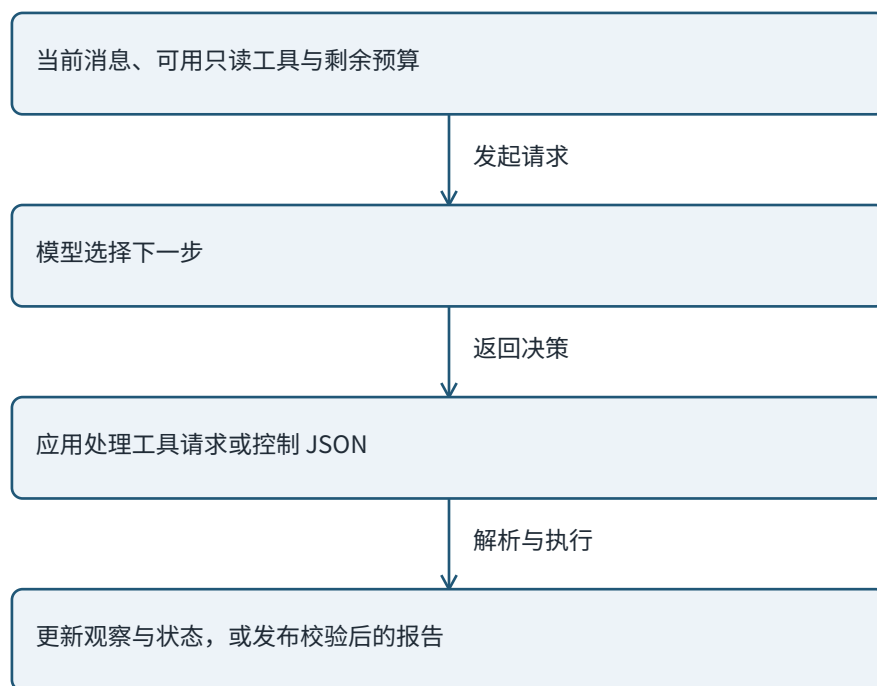


图 1-2 · 一轮 Agent 决策；非终止状态继续以更新后的消息进入下一轮

源码摘录：orchestrator.run 中的最小循环

```
while state.phase not in {InvestigationPhase.COMPLETED, InvestigationPhase.STOPPED,
                        InvestigationPhase.AWAITING_APPROVAL}:
    state = self.advance(state)
```

源码定位：src/ailab_ops/investigation/orchestrator.py · run

advance 的单位是一轮模型决策，不是一条工具调用。某轮可以返回多个只读请求，也可以只更新计划、不调用任何工具。因而 steps_used 与工具数量不能画等号。状态机约束“现在允许做什么”和“什么时候结束”，工具选择则由模型决定。API/CLI 的 runtime 在这个基本循环外增加异步准入和使用量结算；它不会先替模型裁决根因。

阅读 advance 时只抓三个分支

第一次读 orchestrator 不需要理解全部错误处理。先找 gateway.complete，它是模型边界；再找 response.tool_calls，它决定是否处理工具请求；最后找 _control，它负责没有工具请求时的结构化控制输出。理解这三个位置，就能解释一轮是如何闭合的，再逐渐补上预算、重复调用和审批。

源码摘录：工具请求与控制对象的分流

```
if response.tool_calls:
    if session.report_failures:
        return self._stop(session, "report_validation_failed")
    for call in response.tool_calls:
        if self._stop_if_exhausted(session, include_steps=False):
            return state
        self._read_tool(session, call)
        if self._stop_if_exhausted(session, include_steps=False):
            return state
    else:
        self._control(session, response.content)
```

源码定位：src/ailab_ops/investigation/orchestrator.py · advance

这段代码也解释了为什么“模型会用工具”不等于“系统已经可靠”。每次执行前后都要检查停止条件；report 修正阶段不能无边界地再开新调查；工具请求必须经过授权。提示词能提出期望，分支代码才能约束实际行为。第 2 课会把这些保护逐项拆开。

章节关联：第 2 课 · 状态、预算、错误回喂和终止条件

适用边界：何时只需一个函数

若任务只是读取已知 case 的状态，直接调用 get_case_snapshot 就足够。若调查步骤永远固定且规则可以明确写完，普通工作流也可能更合适。Agent 带来的灵活性同时带来不确定输出、额外调用和评测成本；适用性要由任务是否需要基于观察重新选择步骤来判断，不能由工具数量或循环行数判断。

本项目选择一个 Agent 加显式控制层，便于看清责任。把调查拆成多个角色并不会自动提高质量；更多角色意味着更多消息、协作状态和错误来源。先能解释一个会话中的观察与决策，再考虑是否需要独立审查角色，是更容易验证的学习顺序。

1.7 项目位置：沿最短真实调用链阅读

入口汇合到 runtime

CLI 的 investigate 子命令由 cmd_investigate 处理。它读取配置，可用 --mode 覆盖模型模式，建立 runtime，再通过 asyncio.run 调用一次异步调查，最后输出 JSON。API 的 POST /v2/investigations 接收 InvestigationBody，也调用同一个 runtime.investigate。因此，网页、HTTP 和终端展示形式不同，调查控制逻辑可以复用。

源码摘录：CLI 调查调用与退出状态

```
result = asyncio.run(rt.investigate(case_id=args.case, question=args.question,
    max_steps=args.max_steps, max_tokens=args.max_tokens, deadline_s=args.deadline_s))
print(json.dumps(result, ensure_ascii=False, indent=2))
return 0 if result["phase"] == "completed" else 2
```

源码定位：src/ailab_ops/cli.py · cmd_investigate

退出码也有含义：completed 返回 0，其他调查阶段返回 2。排查演示时不要只看有没有打印 JSON，还要查看 phase、stop_reason 和 error。JSON 输出能保留停止后的已有证据；报告缺失不应该被解释成程序已经完成判断。

源码定位：src/ailab_ops/serving/v2.py · InvestigationBody

模型网关与工具注册中心在此相遇

runtime.investigate 先 load_case，以 build_case_registry 创建当前世界的工具，再建立 InvestigationOrchestrator。每次推进模型决策前，runtime 处理使用量与并发准入。orchestrator 把消息和只读工具 spec 交给 gateway.complete；online 网关请求真实模型服务，replay 网关查找对应的录制条目。两者返回相同的 LLMResponse 数据结构，上层继续走同一解析、证据和报告流程。

阅读次序	源码路径与符号	本次先看什么
1	src/ailab_ops/cli.py · cmd_investigate	模式覆盖与调查参数
2	src/ailab_ops/runtime.py · build_runtime	默认建立 V2 runtime
3	src/ailab_ops/v2_runtime.py · investigate / _advance	载入案例、构造控制层、准入后逐步推进
4	src/ailab_ops/investigation/orchestrator.py · advance	模型请求、工具分支和控制分支
5	src/ailab_ops/models/openai.py · complete_with_control	在线 HTTP 请求与响应适配
6	src/ailab_ops/tools/cases.py · build_case_registry	返回观察，不直接返回根因标签
7	src/ailab_ops/evidence/validation.py · validate_report	引用结构校验

源码定位：src/ailab_ops/runtime.py · build_runtime

源码定位：src/ailab_ops/v2_runtime.py · InvestigationRuntime

这里调用真实模型的主路径，和教材演示工具的真实程度是两个维度：模型可以在线，数据仍可来自教学 fixture；未来工具可以读取真实日志，也仍须注明模型是否在线。给每个维度单独标注来源，能防止把一次 API 请求成功包装成已完成生产集成。

在线请求的接口约定

build_model_gateway 在 online 模式检查 llm_base_url、llm_model 和 llm_api_key，拒绝空配置、占位 key EMPTY 和旧 mock 模型名。OpenAIModelGateway 将 base_url 去掉末尾斜杠，再追加 /chat/completions；所以 base_url 通常已经包含 /v1，不应再次追加完整 chat 路径。网关请求流式响应，并将工具调用参数碎片组装为完整 ToolCall；兼容的普通 JSON 响应也可解析。

在线配置与命令示例；占位值需在本地替换，密钥不写入教材或仓库

```
export AILAB_LLM_BASE_URL=https://YOUR_GATEWAY/v1
export AILAB_LLM_MODEL=YOUR_TOOL_CALLING_MODEL
export AILAB_LLM_API_KEY=YOUR_PRIVATE_KEY
export AILAB_MODEL_MODE=online
PYTHONPATH=src python3 -m ailab_ops.cli investigate \
    --case case-gpu-assert --mode online
```

源码定位：src/ailab_ops/models/__init__.py · build_model_gateway

字段示例不是可直接执行的实测环境。配置好相应环境变量后，可以运行本章演示命令并把模式改为 online。实际成功还依赖供应商工具调用兼容性、网络、额度和模型输出质量。本章没有用某家真实模型跑出这些测量值，也不报告在线准确率或费用；下一节的数字来自当前代码执行人工编写回放。

1.8 真实运行证据：一次完整的人工编写回放

回放重现的是系统路径

authored replay，中文称人工编写回放，表示响应由作者预先编写，并非真实模型生成后录下的能力样本。默认录制文件 data/v2/replays/investigations.jsonl 中的 response.model 为 authored-simulation-v2。ReplayModelGateway 的 model 名是 recorded-replay，两者分别标识网关与条目响应。离线演示能验证调用链、消息关联、证据登记和界面展示，不能证明模型学会了排障。

replay 根据规范化后的完整消息与可用工具名称计算哈希，精确查找记录；不会用相似问题、顺序位置或规则推理填补缺失响应。修改 question、工具集或会进入消息的观察，都可能造成 replay_miss。预算数字本身不直接参与这个哈希，但过低预算会使调查提前停止。这个边界让离线演示可以诚实地回答：哪些输入被录制了，哪些没有。

源码摘录：严格回放未命中时直接报错

```
key = canonical_request_hash(messages, [tool.name for tool in tools or []])
if key not in self._responses:
    raise ReplayMissError(key)
```

源码定位：src/ailab_ops/models/replay.py · complete

可复跑的命令与出处

以下命令从仓库根目录执行，要求 Python 3.10 或更高版本以及项目依赖。先安装为 editable package，便于 ailab-ops 找到本仓库源代码与数据。没有 GPU 和 API Key 也可以运行这次离线调查；依赖若已安装，可以直接执行第二条。

本章回放命令；也可使用已安装的 ailab-ops investigate

```
python3 -m pip install -e ".[dev]"
PYTHONPATH=src python3 -m ailab_ops.cli investigate \
    --case case-gpu-assert --mode replay
```

本章数据核对于 2026-10-08，基于代码基线 0bd1dad、上述 CLI 命令、data/v2/cases/case-gpu-assert.json 与 data/v2/replays/investigations.jsonl。tests/test_course_content.py 会重跑实际 CLI，对下面的状态摘要进行比对。session_id、trace_id 和运行时间由执行时生成，故不作为可重复的固定数字。

一次回放的状态摘要：由 CLI JSON 选取字段，非完整返回体

```
{
  "mode": "replay",
  "phase": "completed",
  "steps_used": 5,
  "tokens_used": 1000,
  "evidence_count": 4,
  "root_cause": "gpu_device_assert",
  "confidence": 0.9,
  "tool_calls": [
    "get_case_snapshot",
    "get_case_logs",
    "get_case_metrics",
    "search_runbooks"
  ]
}
```

steps_used 为 5，因为回放有五轮模型响应；工具共调用四次，产生四份证据。tokens_used 为 1000，是条目中预置 usage 累加：每轮输入 120、输出 80，共五轮。这些数字不是模型实际分词统计、不是供应商计费，也不适合用于估算在线延迟。confidence 为预写报告中的 0.9，不表示一组在线故障的 90% 正确率。

五轮响应逐轮读

轮次	响应类型与具体工作	程序处理后的结果
1	plan：读取背景、比较观察、检索、引用报告	更新 plan；证据仍为空
2	native tool_calls：snapshot、logs、metrics	依次读取三类素材；新增三份证据
3	native tool_calls：search_runbooks, top_k=1	用已观察到的错误查询；新增一份 Runbook 证据
4	hypotheses：rank 2 本地断言早于 peer 超时	保存一条候选假设及支持证据
5	report：gpu_device_assert 与未知项	校验引用后 phase=completed

第二轮一次请求了三个工具，控制层按响应顺序执行；这说明一个 step 可以携带批量读取，不表示这些工具在当前实现中并行执行。第三轮查询取自己观察的 rank 2 WARNING/ERROR 文本，包含 device-side assert 与索引断言，命中 GPU device assertion investigation。这条检索比只有“任务为什么失败”更有明确线索，但本章不把一次命中称作检索质量评测。

源码定位：data/v2/knowledge/runbooks/gpu-assert.md

第四轮只记录了一条 hypothesis。架构支持多个候选假设并保留反驳证据，当前 authored replay 却没有演示两条假设竞争后的更新。阅读时必须区分字段支持的能力与本次素材呈现的过程。可以在课后以显存耗尽与网络故障为候选练习比较证据；这些是阅读推演，不是本条回放实际输出。

报告结论与结论的止点

报告的 root_cause 是 gpu_device_assert，summary 表示 rank 2 的本地索引断言先于同伴集体通信超时。两条重要主张分别说明断言的先后关系与采样未显示容量耗尽，每条主张都引用本次收集的四份证据。引用存在且属于会话，但如此宽的引用集合还未细分每条主张的最相关原文。后续改进可以缩小引用粒度；不能因为引用校验通过就宣称语义已被自动证明。

回放报告字段摘录：保留未知项与后续检查建议

```
"unknowns": [
  "The offending input and precise kernel origin remain unproven."
],
"recommendations": [
  "Retain all worker logs and inspect the embedding input indices."
]
```

这条未知项是报告的重要组成部分：具体出错输入和精确内核起点尚未证明。合理行动是保留所有 worker 日志并检查 embedding 输入索引，而不是在未定位时武断降低 batch size。回放结束时 approvals 为空，也没有执行修复。completed 表示这次调查发布了合法报告，不是训练任务已经恢复。

1.9 常见错误：为每个推断找到边界

把症状当根因

反例是只读取 rank 0 的尾部日志，看到 NCCL timeout 就写“网络故障”。案例中 rank 2 的本地错误早十分钟出现，launcher 也记录先观察到 rank 2 退出。这支持本地错误在前、peer 等待在后的解释。若要把网络列为根因，还需要独立的接口错误、链路异常或通信侧观察。最后一条报错的位置不是因果证据。

把一段经验当当前事实

Runbook 解释索引断言的一般调查方法。若模型引用这篇文档后声称“已发现输入中的第 18 条 token 越界”，它跨过了事实缺口：当前工具没有返回输入 payload，step=18 的普通训练日志也不等于已定位坏样本。领域经验能提出检验方向，当前案例观察才能证明本次发生了什么。

把结构化输出当安全执行

模型返回 `{"tool": "restart_job"}` 不代表系统应执行它。工具名称必须存在，参数必须符合约定，策略必须允许；写动作还必须经历独立审批与模拟执行。JSON 的精确格式只能帮助程序读懂请求，不能赋予权限。直接将模型文本交给 `eval` 或 `shell`，会跳过这套受控工具边界，本项目没有这种执行路径。

把回放成功当成模型能力

这次五轮响应由作者提前编写，工具结果由当前代码真实读取。它们一起走通了系统，却没有测试在线模型是否会选择这些工具、是否会在日志缺失时拒绝猜测。更不能用 1000 个预置 token 推导在线成本。在线能力需要独立案例、实际模型运行、重复测量和失败归因。

章节关联：第 5 课 · 分层评测、独立标签和重复运行

把调查结束当故障修复

如果界面显示 `completed`，只能说明调查完成。报告可以建议保留输入或申请日志导出；审批即使通过，也只会在教学环境模拟动作。项目展示了企业系统需要的责任边界，尚不包含对真实集群的写操作，也不应输出“线上已经重启成功”。

1.10 本章总结与课后阅读

本章的调用链从用户目标出发，经 `runtime` 建立案例和工具，由 `orchestrator` 维护消息并请求模型。模型提出工具请求或结构化控制对象；应用执行获准的读取，把观察登记为证据并回喂，最后校验报告结构与引用。LLM 提供判断的灵活性，程序提供行为的边界。每一轮都可以用“当前输入—模型请求—实际观察—状态变化”来解释。

GPU assert 案例说明了这种解释的价值：`peer` 超时并非最早观察，采样内存并不证明所有时间的内存状态，索引断言也尚未定位具体样本。报告表达一个有来源的当前结论，同时保留下一步调查。这比只记住 `gpu_device_assert` 字符串更接近项目真正要教的能力。

课后阅读路线

1. 先读 `data/v2/cases/case-gpu-assert.json`，用时间、rank 和 telemetry 说明重建观察，不先看结论。
2. 再读 `src/ailab_ops/investigation/prompts.py` 与 `llm/base.py`，对照四种消息角色，理解模型得到的协议。
3. 沿 `cli.py`、`runtime.py`、`v2_runtime.py` 到 `investigation/orchestrator.py`，定位 `start`、`advance`、`_read_tool`、`_tool_feedback` 和 `_report`。
4. 最后读 `models/replay.py` 与默认录制文件，确认本章五轮响应如何匹配；需要更改问题时，先理解 `replay_miss` 的原因。

源码定位：`src/ailab_ops/investigation/prompts.py`

模仿方向：从一张调查记录表开始

课后可以运行已有回放，把每轮的响应类型、工具名称、证据增量和 phase 变化记录成一张表，并和本章逐轮表核对。再打开 Web UI，找到同一次调查的计划、证据、假设和报告，说明每个面板的数据对应哪个字段。这个练习不需要重写代码，却能检验是否真正理解了运行路径。

课后查看调查工作区；使用内置案例和默认问题可匹配回放

```
PYTHONPATH=src python3 -m ailab_ops.cli serve --mode replay
# Open http://127.0.0.1:8080
```

理解记录表后，可在自己的临时分支模仿一个更小的调查任务：只提供背景和两类观察，让模型提出有引用的解释。先写工具输入输出和停止条件，再考虑增加模型调用。若选择在线模式，保存工具轨迹并核对引用；若选择离线演示，明确标注自编响应。模仿的重点是责任关系，而不是复刻全部界面或追求最短代码。

思考题与校对线索

问题	可用于校对的线索
为什么第一轮模型不能引用 rank 2 的具体错误？	日志在工具执行后才加入上下文；硬盘上有数据不等于模型已读过。
五个 step 为什么只有四次工具调用？	plan、hypotheses、report 各占一轮；第二轮是三工具批量读取。
若 logs 工具返回缺失早期日志，还能确认 device assert 吗？	peer 超时不足以证明起点，应报告观察和缺口，避免照搬本案例结论。
confidence=0.9 能否解释成在线模型有90%准确率？	这是人工编写报告中的值，没有独立样本或校准。
报告引用了真实存在的证据，为什么仍需评测？	引用存在性不是证据是否支持主张的语义判断。
online 模式调查成功，是否说明真实平台集成完成？	模型在线与工具数据源分别标注；当前工具仍读取教学案例。

下一章将从这个循环进入控制层：它怎样识别重复调用、处理错误、耗尽预算并保存停止状态。后续章节再讨论知识检索、证据质量、审批、评测和服务。阅读时持续区分观察、推断和执行，就能把各层连成一个可检验的项目。